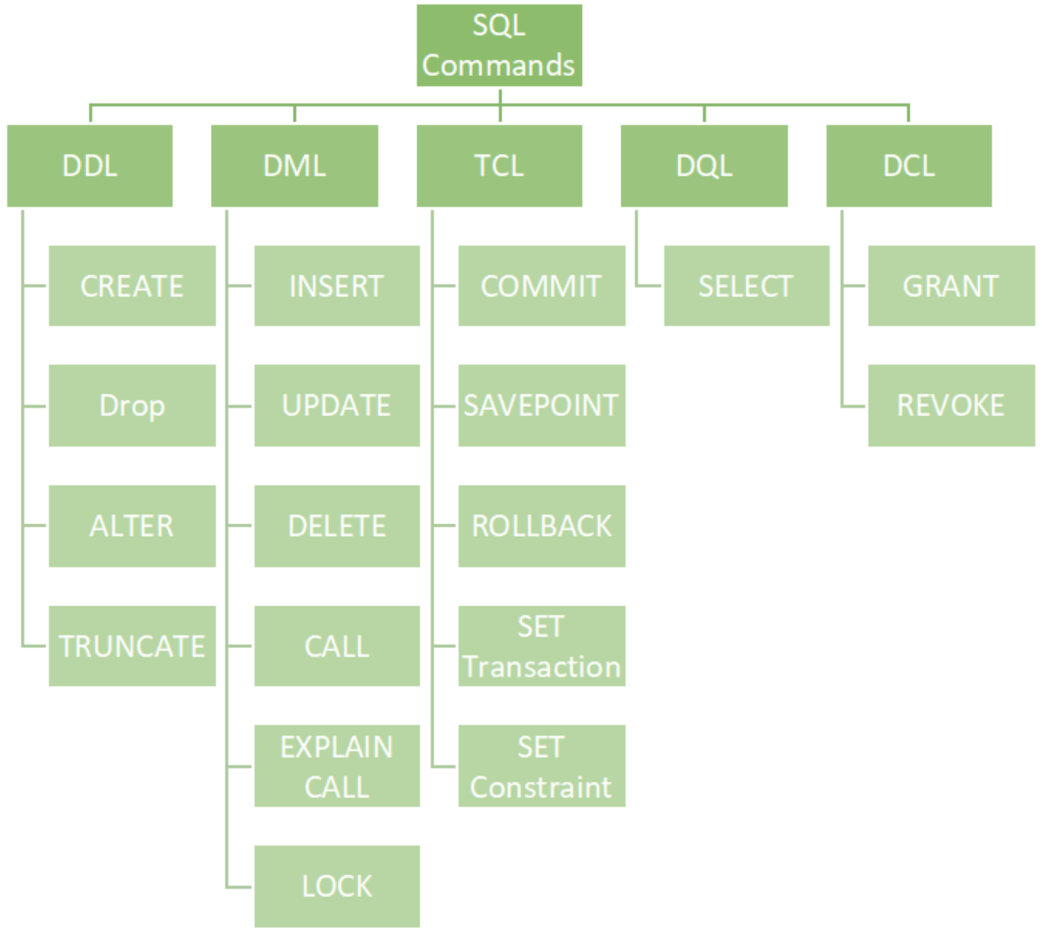
SQL - Structured Query Language



SQL

- **Data Definition Language (DDL)**
  - Defines the structure of database objects
- **Data Manipulation Language (DML)**
  - Works with the actual data inside tables
- **Data Control Language (DCL)**
  - Control the Access and permissions
- **Data Query Langauge (DQL)**
  - Use to read/ query data



Data Definition Language

- **Create**
  - Used to create **database objects**
    - Databases, tables, views, index ...
- **Alter**
  - modifies an **existing database** table. This command can add up additional column, drop existing columns and even change the data type of columns involved in a database table.

```
-- Syntax for table creation
CREATE TABLE [table_name] (column_definitions);
```

```
-- create an employee table with 3 columns
CREATE TABLE employee (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name CHAR(50) NOT NULL,
    last_name CHAR(70) NOT NULL
);
```

```
--Syntax:
ALTER [object_type] [object_name] parameters;
```

```
ALTER TABLE department
ADD PRIMARY KEY(dept_no);
```

```
ALTER TABLE students
ADD COLUMN middle_name VARCHAR(50) AFTER first_name;
```

Data Definition Language

- **Drop**
  - A drop command is used to **delete objects** such as a table, index or view. A DROP statement **cannot be rolled back,** so once an object is destroyed, there's no way to recover it.

```
--Syntax:
DROP [object_type] [object_name];
```

```
-- Drop a table
DROP TABLE employee;
```

- **Truncate**
  - Similar to DROP, the TRUNCATE statement is used to **quickly remove all records** from a table. However, unlike DROP that completely destroys a table, TRUNCATE **preserves it's full structure to be reused later.**

```
--Syntax:
TRUNCATE TABLE [table_name];
```

```
TRUNCATE TABLE Products;
```

Create a table: 2 methods

Method 1:

- Step 1. **create** table structure/schema
  - Like other programming language, you will declare the type of the columns
- Step 2. **insert** data into the table
  - Insert data line by line - for mapping table
  - Bulk insert – for whole chunk of data from file
  - Insert from a select query statement – for data from other tables with manipulations

```
CREATE TABLE students (
    id INT,
    name VARCHAR(50),
    age INT
);
```

```
INSERT INTO students VALUES (1, 'Amy', 20);
```

```
INSERT INTO students (id, name, age)
VALUES
  (1, 'Amy', 20),
  (2, 'Ben', 21),
  (3, 'Cathy', 22);
```

```
INSERT INTO top_students (id, name, score)
SELECT id, name, score
FROM students
WHERE score > 90;
```

- **Method 2**: Create table from a query (doesn't need to create table structure):
  - Syntax: CREATE TABLE <table_name> AS <your select query>

```
CREATE TABLE CustomersFirst10 AS
SELECT *
FROM Customers
WHERE customer_id < 10;
```

Special Tricks:
A quick way to clone table structure, no data!

```
CREATE TABLE CustomerStructure AS
SELECT * FROM Customers
WHERE 1=2;
```

Where 1=2 return no rows

Table already exists?

create **or replace** table <table_name>

Data Manipulation Language

- **Insert**
  - Insert statements are used to input data into tables.
  - The order of data specified should match order of columns.
  - If you don't know the order of columns, specify each column name in the insert statement.

```
— Inserting data matching order.
INSERT INTO TableName VALUES(1, 'Name'),(2, 'Name');

— Inserting data without knowing matching order.
INSERT INTO TableName(Col2, Col1) VALUES('Name', 1),('Name', 2);
```

Data Manipulation Language

- **Update**
  - Update statements are used to change or modify data inside a table.
  - Target specific rows:
    - Specify single row depending on statement.
  - Use unique identifiers to get correct rows.

```
—Update using unique column.
UPDATE TableName SET Name = ‘Tim’ WHERE ID = 2;
```

Data Manipulation Language

- **Delete**
  - Delete statements are used to remove specific rows inside a table.
  - Use unique values to identify the correct rows to delete.

```
— Deleting rows using specific unique values.
DELETE FROM TableName WHERE ID = 1;


— Deleting rows using non-unique values.
DELETE FROM TableName WHERE Name = ‘Jim’
```

Data Manipulation Language - Merge

- **SQL MERGE** allows you to insert, update, or delete data in a target table based on a source table.
- You can specify conditions on which you expect the MERGE statement to insert, update, or delete, etc.
- Key Component:
  - **Source**: Table or query that provides the data.
  - **Target**: The table that will be modified (insert, update, delete).
  - **Matching Condition**: Defines how rows from the source are matched with the target.

```
MERGE INTO target_table AS target
USING source_table AS source
ON target.id = source.id
WHEN MATCHED THEN
    UPDATE SET target.value = source.value
WHEN NOT MATCHED THEN
    INSERT (id, value) VALUES (source.id, source.value);
```

WHEN MATCHED: Defines the action when a row from the source matches a row in the target.
WHEN NOT MATCHED: Defines what happens when there is no match (insert new data)

Data Manipulation Language - Merge

1. **Update**: Modify existing rows in the target table when there's a match.

2. **Insert**: Add new rows to the target table when there is no match.

3. **Delete**: Optionally delete rows in the target table (using `WHEN MATCHED AND <condition>`).
    1. Like (WHEN MATCH AND quality = 0 THEN DELETE)

Data Control Language

- **DCL – Data Control Language.**

- DCL is syntax used to control what permission a user can have.

- You can create Roles that users can be grouped into a specific permission there.

- You can choose what tables someone in a role can access and even what statements can be performed.

Data Control Language

- **Grant**
  - Used to “grant” or give permissions to a user or role.
  - You can check out https://dev.mysql.com/doc/refman/8.0/en/grant.html for all privileges.

- **Revoke**
  - Revoke is used to take away permissions that are previously Granted.
  - You can check out https://dev.mysql.com/doc/refman/8.0/en/grant.html for all privileges.

```
— Grant create table permission to a role (at the database level).
GRANT CREATE ON DatabaseName TO RoleName;

— Grant select and insert permissions to a role (at the table level).
GRANT SELECT, INSERT ON TableName TO RoleName;
```

```
REVOKE [GRANT OPTION FOR] [Permission]
ON [object] FROM [user] [CASCADE]

REVOKE SELECT ON HR.Employees
FROM Joe
```

Data Query Language

- **Basic parts of query**
  - SELECT: Pick up which column of data you'd like to fetch.
  - FROM: Select which table or data set to read from.
  - WHERE: **filter** rows based on conditions.
    - WHERE = your conditions to narrow down results
    - Can use Operators: IN, OR, AND
- **With grouping and sorting**
  - GROUP BY – Used to combine same values in column.
  - HAVING – after group by, filter conditions for aggregate only.
  - ORDER BY – sort results by a column (DESC, ASC(default))

Having:

```
SELECT department, COUNT(*)
FROM Employees
GROUP BY department
HAVING COUNT(*) > 10;
```

SELECT Clause

Attribute	**Example**	**Explanation**
*	*	All attributes from all relations
<table name>.*	FREQUENTS.*	All the attributes from relation <table name>
<alias name>.*	f.*	All the attributes from the relation aliased to f
Attribute list	d.lastName, d.firstName	Only the specified attributes
<Math equation>	1 + 3	Evaluates the expression
<constant>	'CPA' 3	Returns the specified constant

Attribute	Example	Explanation
<function>	NOW() CONCAT(<attribute and string constants) COALESCE(<attributes and constants>)	Current datetime Concatenates the values Returns the first non-NULL argument

Functions can be used in SELECT statement.

https://dev.mysql.com/doc/refman/8.0/en/built-in-function-reference.html

AS

- The AS command is used to create a more meaningful name by renaming a column or table with alias.
- Optional. Only exists for the duration of the query.

```
SELECT first_name AS 'First Name', last_name AS 'Last Name'
FROM employees;
```

```
SELECT e.first_name, d.department_name
FROM employees AS e
JOIN departments AS d ON e.department_id = d.id;
```

Where Clause

```
1. <attribute> = [value]
2. <attribute> BETWEEN [value1] AND [value2]
3. <attribute> IN ([value1], [value2], ...)
4. <attribute> LIKE 'SST%'
5. <attribute> LIKE 'SST_'
6. <attribute> IS NULL AND [attribute] IS NOT NULL
7. Logical combinations with AND and OR
8. Conditional Operators >, <, =, !=
9. Subqueries ...
```

- Conditional operators

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<> or !=	Not equal. In some databases, the `!=` is used to compare values which are not equal.
BETWEEN	Between some range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

SQL GROUP BY

- The GROUP BY statement in SQL is used to **arrange identical data into groups**. This statement is often used with aggregate functions **(COUNT, SUM, MAX, MIN, AVG)** to perform an operation on each group of data.

- Use of GROUP BY:
  - **Aggregating Data**: To calculate aggregate values like total sales, average price, maximum or minimum value within each group.
  - **Reporting and Analysis**: For creating reports that summarize data in a meaningful way, such as sales per region, average salary by department, or total hours worked per project.
  - **Data Organization**: Helps in organizing data in a structured format, making it easier to understand and analyze.

```
SELECT department, COUNT(employee_id)
FROM employees
GROUP BY employee_id;  -- ❌
```

- The columns listed in the GROUP BY clause **must also be listed** in the SELECT statement unless they are used in an aggregate function.

```
SELECT MAX(department), COUNT(employee_id)
FROM employees
GROUP BY employee_id;  -- ✅
```

Put inside an aggregate function

```
SELECT department, COUNT(employee_id)
FROM employees
GROUP BY department;  -- ✅
```

Group properly

- You can group by **multiple columns** by separating column names with commas, which allows for more detailed grouping.

```
SELECT department, location, COUNT(*)
FROM Employees
GROUP BY department, location;
```

- The HAVING clause can be used in conjunction with GROUP BY to filter groups based on a condition applied to aggregated data.

Which is correct?

```
SELECT department, COUNT(employee_id)
FROM employees
WHERE COUNT(employee_id) > 10
GROUP BY department;
```

```
SELECT department, COUNT(employee_id)
FROM employees
GROUP BY department
HAVING COUNT(employee_id) > 10;
```

Aggregate Functions

- Aggregate functions are **built-in functions** that used to perform simple statistics ○
  - COUNT(): returns the number of rows that matches a specified criterion.
  - AVG(): returns the average value of a numeric column.
  - SUM(): returns the total sum.
  - MAX()
  - MIN()
- Trick: use MAX() to return ‘NULL’ instead of nothing

```sql
SELECT MAX(X) AS MaxX
FROM tbl
WHERE XID = 1
```

suppose no row has XID = 1

Aggregate Functions

- How does aggregate functions handle NULL value?
  - **COUNT(*)** or **COUNT(1)** gives the total number of records in the table **including NULL values**
  - **COUNT(column_name)** counts rows where the column contains a **Not-NULL value**
  - AVG, MIN, MAX, etc. **ignore NULL values**
  - GROUP BY **includes a row for null**

Order By Clause

- The **ORDER BY** clause is used to sort the result in ascending or descending order.
- ORDER BY clause sorts the result in ascending order by default. To sort the result by descending order, use **DESC**
- The ORDER BY **must** come after WHERE, GROUP BY, and HAVING clause if present in the query.
- Can be used with **LIMIT**
- Might have other names in other databases (TOP)

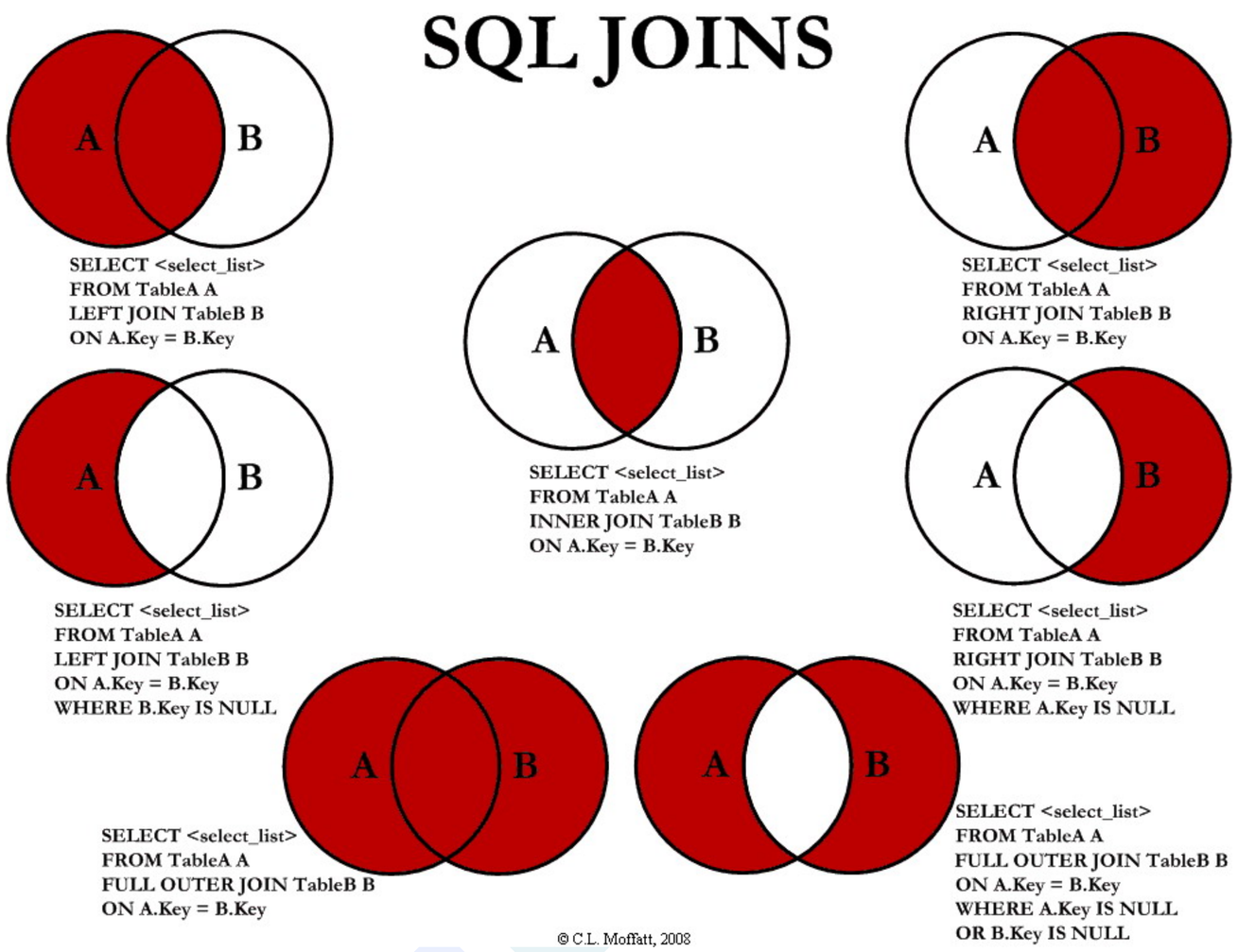
```
SELECT column1, column2,...columnN
FROM table_name
[WHERE]
[GROUP BY]
[HAVING]
ORDER BY column ASC/DESC
```

Joins

- A **JOIN** clause is used to combine rows from 2 or more tables, based on a matching column.
- Uses matching data in specified columns to combine or sort data.
- Matching Columns DO NOT have to have the same name.
- Matching Columns DO NOT need to be keys.
- Scope: table to table, table to view.

Joins

- **Inner Join**
  - Connect on only matching data
  - Will only display values that match on both sides of the table, all others are excluded
- **Left/Right Join**
  - Display the matching data that you'd see from inner join, and all the unique values from the left/right table
- **Full Outer Join** (MySQL doesn't support)
  - Display ALL values, matching and non-matching (In MySQL it can be achieved by using left join and right join)
- **Cross Join**
  - Display every possible combination of all values in the designated columns
- **Self Join**
  - Join a table to itself in some regard


SQL JOINS
A
B
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
© C.L. Moffatt, 2008

Inner Join

- Inner Join returns records that have matching values in both tables.
- Inner Join selects all rows from both tables as long as there is a match between the columns. If there are records in one table that do not have matches in the other table these records will not be shown.

Date	Food
2022-02-23	Ramen
2022-02-24	BBQ
2022-02-26	Sushi
2022-02-28	Thai Curry
2022-02-29	Ramen

Date	Drink
2022-02-22	Boba Milk Tea
2022-02-23	Coffee
2022-02-27	Mach Tea
2022-02-28	Coffee

After INNER JOIN...

Date	Food	Date	Drink
2022-02-23	Ramen	2022-02-23	Coffee
2022-02-28	Thai Curry	2022-02-28	Coffee

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

Left/Right Join

- The LEFT JOIN keyword returns all records from the left table and the matching records from the right table.
- If there is no matching record, NULL will be assigned in the result set.
- It’s a good idea to choose one direction (either LEFT or RIGHT) and use it to maintain consistency.

Date	Food
2022-02-23	Ramen
2022-02-24	BBQ
2022-02-26	Sushi
2022-02-28	Thai Curry
2022-02-29	Ramen

Date	Drink
2022-02-22	Boba Milk Tea
2022-02-23	Coffee
2022-02-27	Mach Tea
2022-02-28	Coffee

After LEFT JOIN...

Date	Food	Date	Drink
2022-02-23	Ramen	2022-02-23	Coffee
2022-02-24	BBQ	NULL	NULL
2022-02-26	Sushi	NULL	NULL
2022-02-28	Thai Curry	2022-02-28	Coffee
2022-02-29	Ramen	NULL	NULL

```sql
SELECT column_name(s)
FROM left_table
LEFT JOIN right_table
ON left_table.column_name = right_table.column_name;
```

Self Join

- Self Join is used when a JOIN is used on the same table.

```
SELECT F.NAME AS EMPLOYEE, MGR.NAME AS MANAGER
FROM FACULTY F JOIN FACULTY MGR
ON F.MGRNETID = MGR.NETID
```

FACULTY

NETID	NAME	MGRNETID
rbm2	Risa	bcd2
abc1	Andre	bcd2
bcd2	Betty	cde4
cde4	Chris	NULL



Results Messages

	EMPLOYEE	MANAGER
1	Andre	Betty
2	Betty	Chris
3	Risa	Betty

Full Outer Join

- Used to match up tuples/rows from different relations
- Includes all the relations from both sides
- If there is no matching tuple, shows NULL

```
SELECT f.name AS employee, m.name AS manager
FROM faculty f
FULL OUTER JOIN faculty m
  ON f.mgrnetid = m.netid;
```

FACULTY

NETID	NAME	MGRNETID
rbm2	Risa	bcd2
abc1	Andre	bcd2
bcd2	Betty	cde4
cde4	Chris	NULL

Results Messages

	EMPLOYEE	MANAGER
1	Andre	Betty
2	Betty	Chris
3	Risa	Betty
4	**Chris**	**Null**

SQL Query Logical Processing Order

- **FROM** – Specifies the tables involved.
- **WHERE** – Filters rows based on a condition.
- **GROUP BY** – Groups rows sharing values of certain columns.
- **HAVING** – Filters groups created by GROUP BY.
- **SELECT** – Chooses columns to display in the result.
- **DISTINCT** – Removes duplicate rows.
- **ORDER BY** – Sorts the results.
- **LIMIT** / **OFFSET** – Restricts the number of rows returned (optional).

SQL Query Logical Processing Order

- Will this query work?

```sql
-- purchases with more total spent lower than 50
SELECT O.name,
       (P.price * O.quantity) AS total_spent
FROM Orders O
JOIN Products P
       ON O.product_id = P.id
WHERE total_spent < 50
ORDER BY total_spent DESC
```

SQL – Subquery/CTE



Subqueries

- A subquery, also known as a **nested query or inner query,** is an SQL query that is embedded within another SQL query. It can be used in various parts of a query, including **SELECT**, **FROM**, **WHERE**, and **HAVING** clauses.
- Subqueries are primarily used for performing operations that require **multiple steps in data processing**. They allow for more flexible and efficient data retrieval, enabling complex operations to be executed in a single query.
- **Benefits**:
  - **Enhancing Data Retrieval**: sometimes the use of standard SQL queries might be limiting or inefficient.
  - **Complex Data Relationships**: when data relationships are complex and require multiple layers of filtering or selection.
  - **Nested Logic**: offer a way to implement nested logic in SQL queries

- Subqueries can be really expensive!!!

Characteristic

- **Return Types**: Subqueries can return **individual values**, **a single row**, **multiple rows**, or even a **table-like structure**.
- **Independent vs. Correlated**: They can be independent of the outer query (standalone) or correlated, where the subquery depends on data from the outer query.
- **Placement Flexibility**: Subqueries can be used in various places within a query, such as:
  - In the SELECT clause, to add columns to the main query.
  - In the FROM clause, to create a derived table that the main query can use.
  - In the WHERE or HAVING clauses, to filter records based on complex conditions.

Independent：

```
SELECT name
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

Correlated:

```
SELECT name
FROM Employees e
WHERE salary > (SELECT AVG(salary) FROM Employees WHERE department = e.department);
```

Single-Row Subqueries

- **Definition**: Those return a single row from the inner query. They’re typically used with single-row comparison operators like =, >, <, <=, >=, and <>.
- **Use Cases**: For instance, finding an employee whose salary is equal to the maximum salary in the department or the value of a column is above or below average.
- Example:

```
SELECT payment_id, amount

FROM Payment

WHERE amount = (SELECT MAX(amount) FROM Payment)
```

Multiple-Row Subqueries

- **Definition**: These return multiple rows and are used with existential or set membership operators like IN, ANY, ALL, or comparisons combined with ANY or ALL.
- **Use Cases**: Useful when the subquery is expected to return more than one row. For example, retrieving all orders that match any of the order IDs in a list.
- **Alternative**: A simple inner-join or a theta-join can often be used instead.
- **Example**:
- The query below finds how many cities are located in a country that starts with a ‘B’.

```
SELECT count(*) AS NumCities
FROM City
WHERE country_id
IN (SELECT country_id FROM Country WHERE country LIKE 'B%')
```

Correlated Subqueries

- **Definition**: A correlated subquery is one that **uses values from the outer query.** It is executed repeatedly, once for each row that might be selected by the outer query.
- **Use Cases**: Effective in scenarios where each row in the outer query may need different results from the subquery.
- For instance, finding employees who earn more than the average salary in their respective departments.

```
SELECT staff_id, amount

FROM Payment AS P

WHERE amount > (SELECT AVG(amount) FROM Payment WHERE staff_id = P.staff_id)
```

Subquery as a Derived Table

- **Definition:** This involves using a **subquery in the FROM clause** of the main query. The result of the subquery acts as a **temporary table**.
- **Use Cases:** Useful for creating complex joins and aggregations. For example, summarizing data in a subquery before joining it to another table.
- **Example:**

```
SELECT C.customer_id, C.last_name, P.total_amount
FROM Customer AS C
JOIN (SELECT customer_id, sum(amount) AS `total_amount` FROM Payment GROUP BY customer_id)
AS P
ON (C.customer_id = P.customer_id)
```

customer_id	total_amount
1	250.00
2	500.00

customer_id	last_name	total_amount
1	Smith	250.00
2	Johnson	500.00

SQL Common Table Expressions (CTEs)

The SQL **WITH statement**, also known as Common Table Expressions **(CTEs),** is a useful feature for **creating temporary result sets** that can be referenced within a SELECT, INSERT, UPDATE, or DELETE statement.

```
WITH CTE_Name AS
(
 Your SQL query here
)
SELECT * FROM CTE_Name;
```

CTE_Name is a temporary result set that you define within the WITH clause. This result set can then be used in the main SQL query that follows in place of any table.

CTE Use Case 1: Simplify Complex Joins

Suppose you want to find films with the **highest rental rates** in each rating category. This might involve complex joins and aggregations. Using a CTE, you can simplify it:

```
SELECT f.film_id, f.title, f.rating, f.rental_rate
FROM Film f
INNER JOIN (
    SELECT rating, MAX(rental_rate) AS MaxRate
    FROM Film
    GROUP BY rating
) h
ON f.rating = h.rating
   AND f.rental_rate = h.MaxRate;
```

```
WITH HighestRentalRates AS
( SELECT rating, MAX(rental_rate) AS MaxRate
FROM Film
GROUP BY rating
 )
SELECT f.film_id, f.title, f.rating, f.rental_rate
FROM Film f
INNER JOIN HighestRentalRates h
ON (f.rating = h.rating AND f.rental_rate = h.MaxRate);
```

CTE result:

Rating	MaxRate
G	4.99
PG	5.99
R	6.99

Final result:

film_id	title	rating	rental_rate
101	Fun Cartoon	G	4.99
205	Family Adventure	PG	5.99
309	Action Thriller	R	6.99

CTE Use Case 2: Recursive CTE（optional)

Assuming we have a table film_category with a hierarchical structure(tree-like), the query might look

category_id	category_name	parent_category_id
1	Movies	NULL
2	Action	1
3	Comedy	1
4	Superhero	2
5	Martial Arts	2
6	Romantic Comedy	3
7	Slapstick	3

```
Movies (id 1)
├── Action (id 2)
│   ├── Superhero (id 4)
│   └── Martial Arts (id 5)
└── Comedy (id 3)
    ├── Romantic Comedy (id 6)
    └── Slapstick (id 7)
```

```
WITH RECURSIVE SubCategory AS
(
-- Anchor query (base case, non-recursive part)
SELECT category_id, parent_category_id
FROM film_category
WHERE parent_category_id IS NULL

UNION ALL

-- Recursive query (refers to SubCategory)
SELECT fc.category_id, sc.parent_category_id
FROM film_category fc
INNER JOIN SubCategory sc
ON sc.parent_category_id = sc.category_id

)

SELECT * FROM SubCategory;
```

Recursive CTE

```
WITH employee_manager_cte AS (
  SELECT
    id,
    name,
    department,
    manager_id,
    manager_name,
    1 AS level
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT
    e.id,
    e.name,
    e.department,
    e.manager_id,
    e.manager_name,
    level + 1
  FROM employees e
  INNER JOIN employee_manager_cte r
    ON e.manager_id = r.id )
```

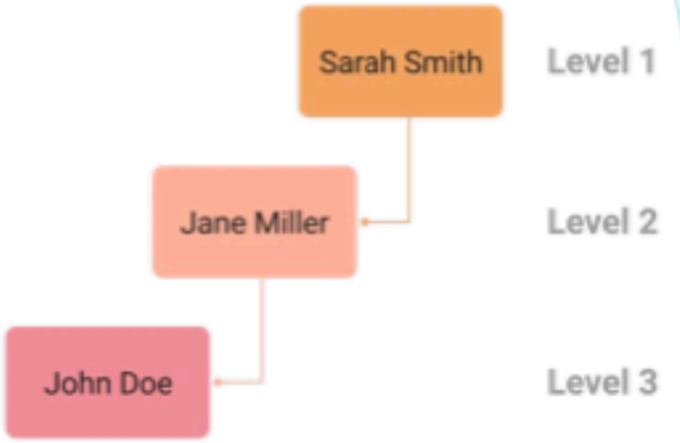
id	name	department	manager_id	manager_name
124	John Doe	IT	135	Jane Miller
135	Jane Miller	HR	146	Sarah Smith
146	Sarah Smith	CEO	NULL	NULL

```
SELECT *
FROM employee_manager_cte;
```

```sql
WITH employee_manager_cte AS (
  SELECT
    id,
    name,
    department,
    manager_id,
    manager_name,
    1 AS level
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT
    e.id,
    e.name,
    e.department,
    e.manager_id,
    e.manager_name,
    level + 1
  FROM employees e
  INNER JOIN employee_manager_cte r
    ON e.manager_id = r.id )
```

anchor member

recursive member



id	name	department	manager_id	manager	level
146	Sarah Smith	CEO	NULL	NULL	1
135	Jane Miller	HR	146	Sarah Smith	2
124	John Doe	IT	135	Jane Miller	3

SQL Window Functions



Window Function

- special functions that perform calculations across rows that are related to the current row **without grouping the rows together.**
- **In a Nutshell**:
- OVER (PARTITION BY col_name ORDER BY col_name ASC/DESC)
  - order by – how you want to order the data, then do the calculation/ranking?
  - partition by – is it important to display the derived data by group?

- row_number(), rank(), dense_rank()
  - Sample data: 80, 90, 90, 100
  - row_number(): 1, 2, 3, 4
  - rank(): 1, 2, 2, 4
  - dense_rank(): 1, 2, 2, 3
- count(), sum(), avg()
- lead(), lag()

Demo code — need to be added

Window Function

- The **PARTITION BY** clause in SQL is a powerful feature used in conjunction with window functions. It allows you to partition a result set into **groups** and perform calculations or apply functions **to each group (partition) independently**.

- **Purpose**: PARTITION BY divides the result set into partitions to which the window function is applied. Instead of treating the entire result set as a single group, it enables calculations across a set of rows that share common properties.

- Use with Window Functions: It’s used with window functions like ROW_NUMBER(), RANK(), SUM(), AVG(), etc. These functions perform a calculation across a set of table rows related to the current row.

Example 1: Ranking Customers by Amount Spent

```
SELECT customer_id, amount,
       RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC) as rank
FROM payment
LIMIT 5;
```

Table 9: 5 records

customer_id	amount	rank
1	9.99	1
1	7.99	2
1	5.99	3
1	5.99	3
1	5.99	3

Example 2: Running Total of Payments by Customer

```
SELECT customer_id, payment_date, amount,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY payment_date) as running_total
FROM payment
```

Table 10: Displaying records 1 - 10

customer_id	payment_date	amount	running_total
1	2005-05-25 11:30:37	2.99	2.99
1	2005-05-28 10:35:23	0.99	3.98
1	2005-06-15 00:54:12	5.99	9.97
1	2005-06-15 18:02:53	0.99	10.96
1	2005-06-15 21:08:46	9.99	20.95
1	2005-06-16 15:18:57	4.99	25.94
1	2005-06-18 08:41:48	4.99	30.93
1	2005-06-18 13:33:59	0.99	31.92
1	2005-06-21 06:24:45	3.99	35.91
1	2005-07-08 03:17:05	5.99	41.90

A classic SQL problem

- **Island and gaps problem**
  - https://mattboegner.com/improve-your-sql-skills-master-the-gaps-islands-problem/
  - https://binhhoang.io/blog/gaps-and-islands/